

Run the model pipeline on real data and report row counts

Context

The three model scripts already persist predictions. `models/baseline.py` line 297 defines `upsert_predictions`; `models/gbm.py` line 456 and `models/ensemble.py` line 432 both call it. Nothing needs to be built here.

The dashboard is currently showing the eight fake `demo_` players from `scripts/seed_demo.py` because that seeder is the only thing that has actually been run against the database. The real pipelines have never been executed on real data.

This card is an execution and verification task. Do not add features, do not refactor, and do not change model logic.

Tasks

1. Report the current row counts, before changing anything, for: `matches`, `players`, `player_game_features`, `odds`, and `player_prop_predictions`. For each, split the count into real rows and rows where `player_id LIKE 'demo_%'`. Paste the actual query output.
2. Confirm the prerequisites are satisfied. `player_game_features` must contain real ATP rows for the stat targets `aces`, `double_faults`, and `service_games` at `feature_version = 'tennis_v1'`. If it is empty or demo-only, stop, report exactly that, and do not proceed to step 3 — the features must be built first and that is a different task.
3. Run the baseline for all three targets against the real data, not in dry-run mode. Report the row count written and the `modelversion` values produced.
4. Run the GBM for all three targets, not in dry-run mode. Report row counts written per `modelversion`. Also report the shuffled-target check output that `run_gbm` prints — if the shuffled-target error is not clearly worse than the ordinary error, something is leaking and you should say so loudly rather than continue.
5. Run the ensemble for all three targets, not in dry-run mode. Report row counts and the weights used, and state explicitly whether those weights were fitted or read from config.
6. Query `player_prop_predictions` grouped by `modelversion` and report the final counts, plus the min and max `predictiontimestamp` per version. Confirm no pre-existing `modelversion` was overwritten — if a version that existed in step 1 has a changed row count, say so.
7. Select ten rows from `vw_fact_player_prop_odds` for real players and paste them. Confirm that `player` shows real names, that `prediction`, `lowerci`, and `upperci` are populated, and that `line`, `edge`, and `side` are NULL. All three NULLs are expected and correct, because the view LEFT JOINs odds and there are no ATP odds yet.

Do not

Do not delete or modify demo rows in this card. Do not touch the reporting view. Do not create a new migration. Do not run `seed_demo.py`.

Report

Report which items you completed and explicitly list any you did not do and why. Do not silently skip items. If step 2 stopped you, that is a valid and useful outcome — report it as such rather than working around it.

What to look for

The single number that matters is the count of real, non-demo rows in `player_prop_predictions` at step 6. If that is greater than zero and step 7 shows real player names, the dashboard problem is solved by this card alone.

Step 4's shuffled-target check is the leakage guard: it retrains on deliberately scrambled targets, and a model that still performs well on nonsense targets is reading something it should not. Step 6 is the model-version guard: because the table has a UNIQUE constraint on `(player_id, match_id, prop_type, modelversion)`, re-running with an unchanged version updates rows in place rather than appending, which quietly destroys the ability to compare versions.